

NumPy Power Tools: Indexing & Advanced Operations

Suman 6 Feb, 2026 At 8:00 PM Notes Recommended Resource



Associated Lectures (1)



Description



Discussions

Session 3.2: NumPy Power Tools: Indexing & Advanced Operations

In-Class Resources

Lecture Notes

Program: AIM101 - Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna x Masai School

Mental Map

Session 3.2: NumPy Power Tools

```
|
+-- A. Fancy Indexing
|   +-- 1D fancy indexing: arr[[0, 2, 5]]
|   +-- 2D fancy indexing: matrix[[rows], [cols]]
|   +-- Row selection: matrix[[0, 2]]
|   +-- Key insight: Order of indices = order of result
|
+-- B. Boolean Masking
|   +-- Creating masks: arr > 5 --> [F, T, T, F, ...]
|   +-- Filtering with masks: arr[mask]
|   +-- np.where(cond) --> indices
|   +-- np.where(cond, true_val, false_val) --> replacement
|   +-- CRITICAL: Compound conditions need parentheses!
|       +-- AND: (arr > 5) & (arr < 10)
|       +-- OR:  (arr > 5) | (arr < 2)
|       +-- NOT: ~(arr > 5)
|
+-- C. Universal Functions (ufuncs)
|   +-- Aggregations: sum, mean, std, min, max
|   +-- axis parameter:
|       |   +-- axis=None --> all elements
```

```

| | +-- axis=0 --> operate DOWN (kills rows)
| | +-- axis=1 --> operate ACROSS (kills columns)
| +-- Math functions: sqrt, exp, log
|
+-- D. Reshaping Operations
| +-- reshape(rows, cols) --> total must match!
| +-- -1 trick: reshape(2, -1) --> auto-calculate
| +-- flatten() --> returns COPY
| +-- ravel() --> returns VIEW
| +-- transpose() or .T --> swap rows/columns
|
+-- E. Views vs Copies
| +-- VIEWS (share memory):
| | +-- Slicing: arr[1:4]
| | +-- reshape(), ravel(), .T
| +-- COPIES (independent):
| | +-- Fancy indexing: arr[[1,3,5]]
| | +-- flatten()
| | +-- .copy() method
| +-- Check with: arr.base is original
|
+-- F. Linear Algebra Basics
+-- Dot product: np.dot(a, b) or a @ b
+-- Element-wise: A * B
+-- Matrix multiplication: A @ B
+-- Rule: (m,n) @ (n,p) --> (m,p)

```

The Big Picture

In Session 3.1, you learned the fundamentals: why NumPy is fast (contiguous memory, compiled C code), how to create arrays, and element-wise operations. Now we unlock NumPy's true power.

This session equips you with **surgical precision** for data manipulation. Fancy indexing lets you grab any elements you want from anywhere in an array. Boolean masking filters data based on conditions in a single line of code. Aggregations with the axis parameter let you compute statistics across rows or columns. Reshaping transforms your data structure to fit any algorithm's requirements.

Why this matters for data science: Every machine learning pipeline involves filtering outliers (boolean masking), calculating feature statistics (aggregations), and reshaping data for model input. The Knight Capital bug that cost \$440 million in 45 minutes stemmed from errors in numerical array operations. Understanding these tools deeply is not optional.

Bridge to Session 3.3: Once you can manipulate arrays, you need to make that manipulation fast. Next session covers vectorization patterns, performance profiling, and an introduction to Numba for when you need even more speed.

Main Content (Condensed)

A. Fancy Indexing

What it is: Selecting non-contiguous elements using an array of indices.

Syntax:

```
arr = np.array([10, 20, 30, 40, 50, 60, 70])

# Select indices 0, 2, 5
arr[[0, 2, 5]]      # Returns: [10, 30, 60]

# ORDER MATTERS!
arr[[5, 0, 2]]      # Returns: [60, 10, 30]

# Can repeat indices
arr[[0, 0, 1, 1]]   # Returns: [10, 10, 20, 20]
```

2D Fancy Indexing:

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

# Select specific elements: (row0, col1) and (row2, col2)
matrix[[0, 2], [1, 2]]  # Returns: [2, 9]

# Select entire rows 0 and 2
matrix[[0, 2]]          # Returns: [[1,2,3], [7,8,9]]
```

Key Insight: The order of your index array determines the order of results.

B. Boolean Masking

Creating masks:

```
scores = np.array([45, 67, 82, 55, 90, 38, 75])
mask = scores >= 60    # [False, True, True, False, True, False, True]
```

Applying masks:

```
passing = scores[mask]      # [67, 82, 90, 75]
passing = scores[scores >= 60] # Shorthand - same result
```

np.where() - Two uses:

```
# Use 1: Find INDICES where condition is True
indices = np.where(scores >= 60)  # Returns: (array([1, 2, 4, 6]),)
                                   # Note: Returns tuple!

# Use 2: Conditional replacement
```

```
labels = np.where(scores >= 60, 'Pass', 'Fail')
# Returns: ['Fail', 'Pass', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass']
```

CRITICAL - Compound Conditions:

```
# WRONG - will error or give unexpected results!
scores > 60 & scores < 90

# CORRECT - must use parentheses!
(scores >= 60) & (scores < 90)    # AND
(scores < 50) | (scores >= 90)    # OR
~(scores >= 60)                  # NOT
```

Why parentheses? The `&` operator has higher precedence than comparison operators. Without parentheses, Python parses `arr > 60 & arr < 90` as `arr > (60 & arr) < 90`.

C. Universal Functions (ufuncs)

Aggregation Functions:

```
grades = np.array([[85, 90, 78, 92],
                  [88, 76, 95, 85],
                  [90, 88, 82, 79]]) # Shape: (3, 4)

np.sum(grades)          # 1028 - all elements
np.mean(grades)         # 85.67 - average of all
np.std(grades)          # Standard deviation
np.min(grades)          # 76
np.max(grades)          # 95
```

The axis Parameter:

axis	Direction	What Disappears	Result Shape
None	All elements	Everything	Single value
0	DOWN columns	Rows disappear	(4,)
1	ACROSS rows	Columns disappear	(3,)

```
# axis=0: sum DOWN each column (rows collapse)
np.sum(grades, axis=0)    # [263, 254, 255, 256] shape (4,)

# axis=1: sum ACROSS each row (columns collapse)
np.sum(grades, axis=1)    # [345, 344, 339] shape (3,)
```

Memory Trick: "The axis you specify is the axis that DISAPPEARS."

- axis=0: Row dimension (3) disappears, left with (4,)

- axis=1: Column dimension (4) disappears, left with (3,)

D. Reshaping Operations

reshape() - Change array dimensions:

```
arr = np.arange(12)    # [0, 1, 2, ..., 11]

arr.reshape(3, 4)      # 3 rows, 4 columns
arr.reshape(4, 3)      # 4 rows, 3 columns
arr.reshape(2, -1)     # 2 rows, auto-calculate columns --> (2, 6)
```

Golden Rule: Total elements must match! $3 \times 4 = 12$ works, $3 \times 5 = 15$ fails.

flatten() vs ravel():

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]])

flat = matrix.flatten() # Returns COPY - changes are independent
rav = matrix.ravel()    # Returns VIEW - changes affect original!

rav[0] = 99
print(matrix[0, 0])     # 99 - original changed!
```

transpose() or .T:

```
matrix = np.array([[1, 2, 3],
                   [4, 5, 6]]) # Shape (2, 3)

matrix.T # Shape (3, 2)
# [[1, 4],
#  [2, 5],
#  [3, 6]]
```

E. Views vs Copies

This is where bugs hide!

Operation	Result	Memory Behavior
<code>arr[1:4]</code> (slicing)	View	Shares memory
<code>arr.reshape(3,4)</code>	View	Shares memory
<code>arr.ravel()</code>	View	Shares memory
<code>arr.T</code>	View	Shares memory

Operation	Result	Memory Behavior
<code>arr[[1,3,5]]</code> (fancy indexing)	Copy	Independent
<code>arr.flatten()</code>	Copy	Independent
<code>arr.copy()</code>	Copy	Independent

Why it matters:

```
arr = np.array([1, 2, 3, 4, 5])

# Slicing creates a VIEW
view = arr[1:4]
view[0] = 99
print(arr)    # [1, 99, 3, 4, 5] - ORIGINAL CHANGED!

# Fancy indexing creates a COPY
copy = arr[[1, 2, 3]]
copy[0] = 99
print(arr)    # [1, 99, 3, 4, 5] - Original unchanged by this
```

How to check:

```
view.base is arr    # True --> it's a view
copy.base is arr    # False --> it's a copy
```

How to force a copy:

```
safe = arr[1:4].copy()    # Now modifications are isolated
```

F. Linear Algebra Basics

Dot Product - combining vectors into a single number:

```
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])

# Two equivalent ways
np.dot(v1, v2)    # 32
v1 @ v2           # 32

# Manual: 1*4 + 2*5 + 3*6 = 4 + 10 + 18 = 32
```

Element-wise vs Matrix Multiplication:

```
A = np.array([[1, 2],
               [3, 4]])
```

```

B = np.array([[5, 6],
              [7, 8]])

# Element-wise multiplication (*)
A * B      # [[5, 12], [21, 32]]

# Matrix multiplication (@)
A @ B      # [[19, 22], [43, 50]]

```

Matrix multiplication rule: For $(m, n) @ (n, p)$, inner dimensions must match. Result is (m, p) .

Key Frameworks

1. Indexing Decision Tree

```

Need to select elements from an array?
|
+-- Is it a contiguous block?
|   +-- YES --> Use slicing: arr[1:4]
|   +-- NO --> Continue...
|
+-- Do you have specific indices?
|   +-- YES --> Use fancy indexing: arr[[1, 3, 5]]
|   +-- NO --> Continue...
|
+-- Is selection based on a condition?
    +-- YES --> Use boolean masking: arr[arr > 5]

```

2. axis Parameter Memory Trick

"The axis you specify is the axis that DISAPPEARS"

Original shape: (3, 4)
 ^ ^
 axis0 axis1

axis=0 --> 3 disappears --> Result: (4,)
 "Operate DOWN, collapse rows"

axis=1 --> 4 disappears --> Result: (3,)
 "Operate ACROSS, collapse columns"

axis=None --> Everything collapses --> Result: single value

3. View vs Copy Checklist

VIEWS (share memory):	COPIES (independent):
-----	-----
Slicing: arr[1:4]	Fancy indexing: arr[[1,3,5]]
reshape()	flatten()

```
ravel()
transpose() / .T
```

```
.copy()
Boolean indexing: arr[arr > 5]
```

Check with: `arr.base` is original

- True --> VIEW
- False (or None) --> COPY

Common Errors Quick Reference

Error Message	Cause	Solution
<code>ValueError: operands could not be broadcast together</code>	Shape mismatch in compound boolean conditions	Add parentheses: <code>(arr > 5) & (arr < 10)</code>
<code>ValueError: cannot reshape array of size X into shape (a, b)</code>	<code>a * b != X</code>	Ensure total elements match
<code>IndexError: index X is out of bounds for axis 0 with size Y</code>	Fancy index $\geq$ array length	Check that all indices $<$ array size
<code>TypeError: ufunc 'bitwise_and' not supported for the input types</code>	Using <code>and</code> instead of <code>&</code>	Use <code>&</code> for NumPy boolean operations
Unexpected data modification	Modified a view, which changed the original	Use <code>.copy()</code> when you need independence

Key Takeaways

Fancy indexing uses index arrays to select any elements in any order. Order of indices determines order of results.

Boolean masking filters arrays by condition. Always wrap compound conditions in parentheses:

```
(cond1) & (cond2) .
```

np.where() has two uses: find indices (`np.where(cond)`) or replace values (`np.where(cond, true_val, false_val)`).

axis parameter: The axis you specify is the axis that disappears. `axis=0` operates DOWN (collapses rows), `axis=1` operates ACROSS (collapses columns).

Reshape golden rule: Total elements must remain constant. Use `-1` for automatic dimension calculation.

Views share memory (slicing, reshape, ravel, transpose). **Copies are independent** (fancy indexing, flatten, `.copy()`). When in doubt, use `.copy()` .

Next Session Preview

Session 3.3: Optimising Numerical Code [EXPOSURE]

You now know how to manipulate arrays. Next session teaches you how to make that manipulation fast.

Topics covered:

- **Vectorization patterns** - replacing slow Python loops with fast NumPy operations
- **Profiling code** - using `%timeit` to identify bottlenecks
- **Introduction to Numba** - just-in-time compilation for when you need C-like speed (awareness level)
- **When to optimize** - premature optimization is the root of all evil, but knowing when to optimize is a valuable skill

Real-world context: In high-frequency trading, algorithms execute millions of operations per second. The Knight Capital disaster showed what happens when numerical code is not properly tested. Performance is not just about speed; it is about correctness under pressure.

Vishlesan i-Hub IIT Patna x Masai School